Artificial Intelligence

Local Search

Conducted by:

Dr. Fazlul Hasan Siddiqui

(fazlul.siddiqui@duet.ac.bd)

Credit: Stuart Russell and Peter Norvig

Local Search

- Only the solution is important - not how you got there.
- Operate on a current solution
- Define a Neighbourhood which they will explore
- Test members of the neighbourhood, and potentially install new current solution.
- The method of choice for optimization problems
- Optimisation: Search for a solution with the lowest (highest) value of an objective function

Local Search – Success stories

- ♢ Routing problems
- ♢ Scheduling (including timetabling, rostering)
- ♢ Location problems
- ♢ Network optimization
- ♢ Layout problems
- ♢ . . . lots and lots of others

Local Search – characteristics

- ♢ Little memory (usually constant)
- ♢ Find reasonable solutions in huge state spaces

Neighbourhood

- The neighbourhood defines a set of *potential successor states*
- Often defined by the an operator that *moves* from one solution to another
- Multiple neighbourhoods possible
- The larger the neighbourhood, the better the chance we will see a global optimum
- . . . but the longer it will take to search it

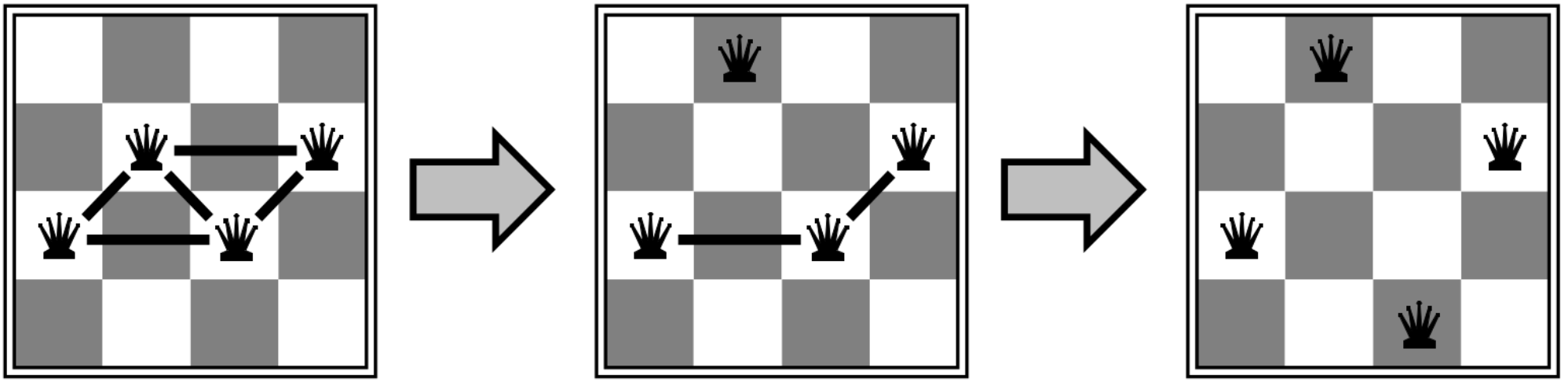
Example: the n-Queens Problem

- ♢ Put n queens on an $n \times n$ chessboard
- ♢ No two queens on the same row, column, or diagonal

Iterative improvement:

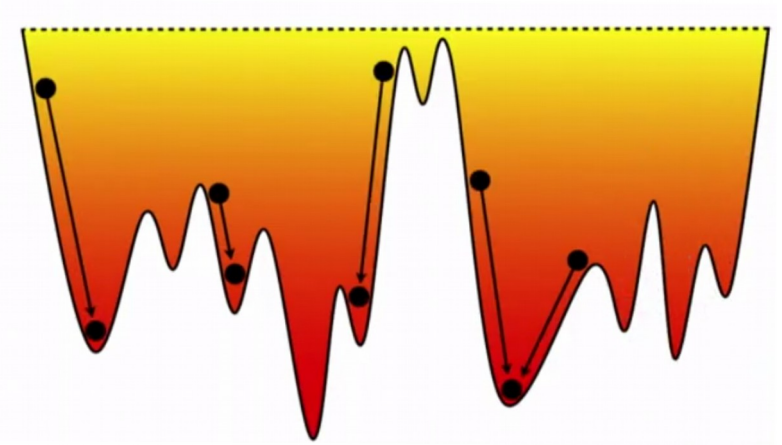
Start with one queen in each column

move a queen to reduce number of conflicts



Even for very large n (e.g., $n = 1\ million$),
this usually finds a solution almost instantly

8-Queens Problem
Local Search:
Iterative Improvement



Total violations
Step count

Travelling Salesman Problem

In this algorithm, the objective is to find a low-cost tour that starts from a city, visits all cities en-route exactly once and ends at the same starting city.

Start

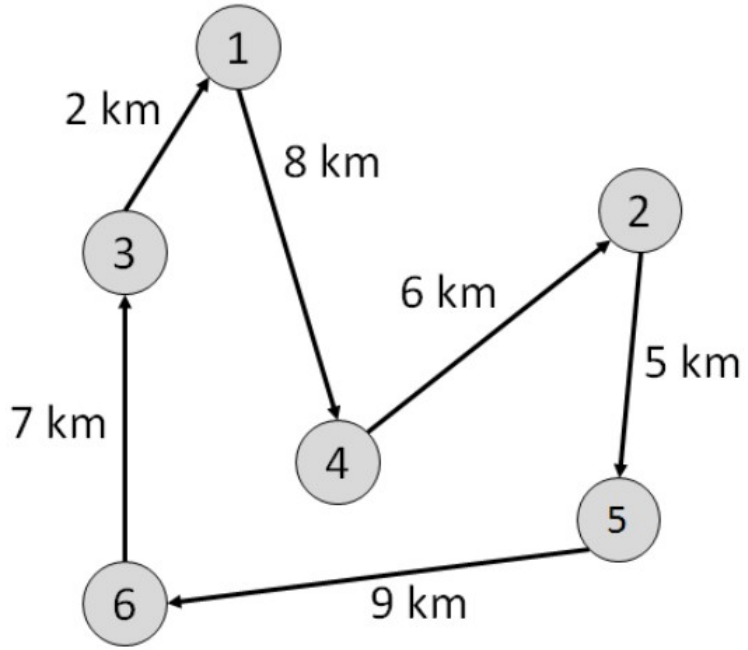
Find out all (n -1)! Possible solutions, where n is the total number of cities.

Determine the minimum cost by finding out the cost of each of these (n -1)!

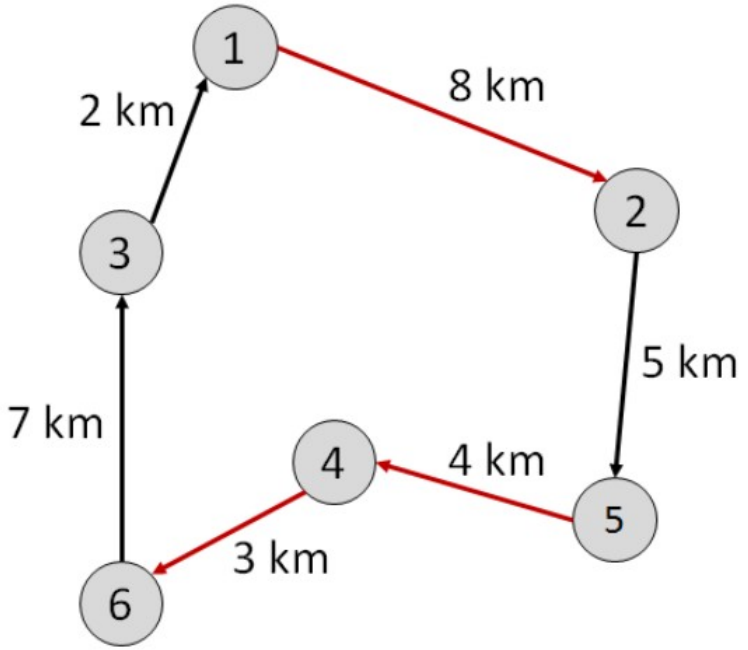
solutions.

Finally, keep the one with the minimum cost.

end

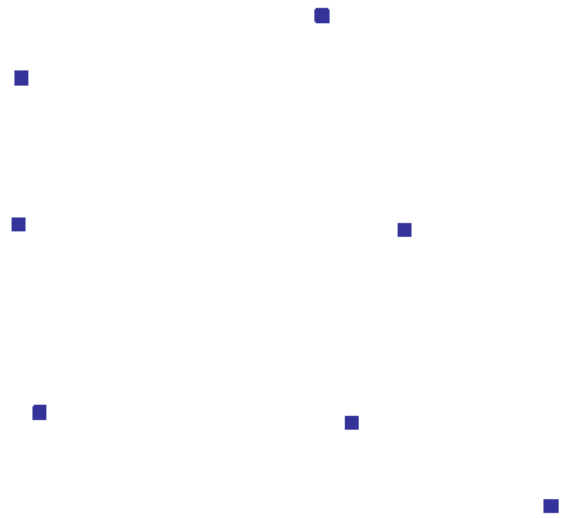


Total Distance = 37km



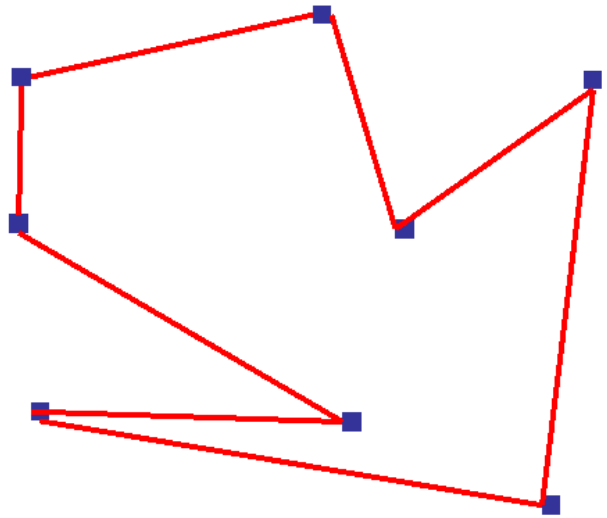
Total Distance = 31km

Travelling Salesman Problem



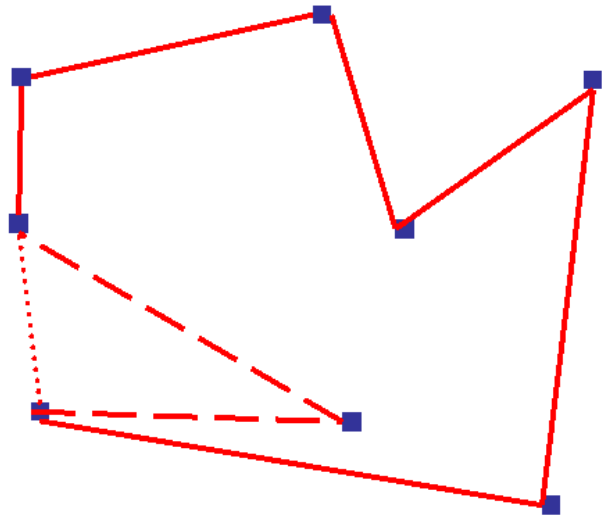
Given a set of points, visit them all and return to the first at minimum cost.

Travelling Salesman Problem



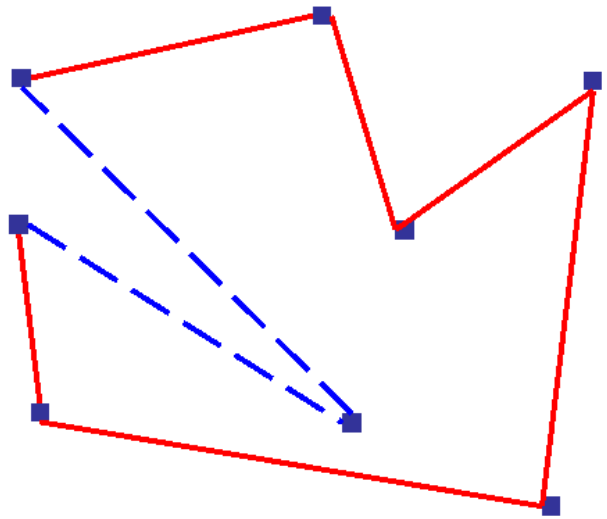
Given a set of points, visit them all and return to the first at minimum cost.

Neighbourhood



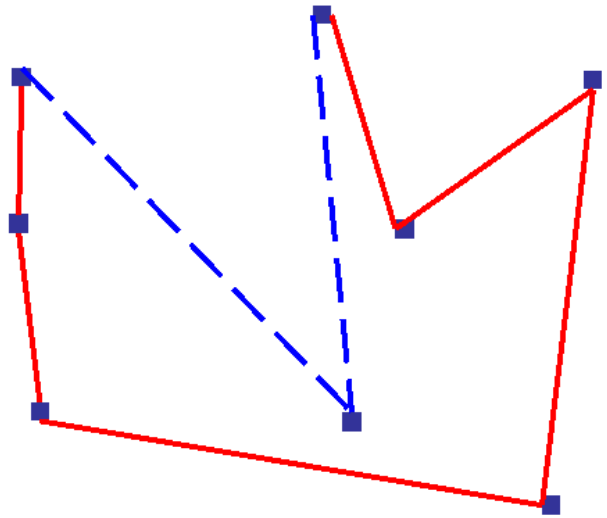
Remove a node. Test inserting it between each pair of nodes. Choose best.

Neighbourhood



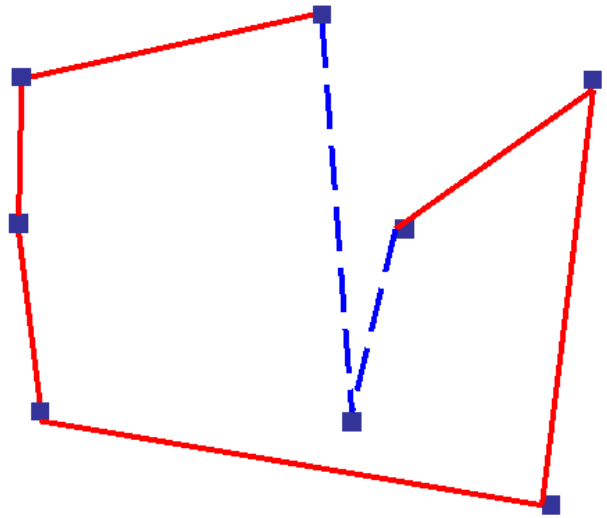
Remove a node. Test inserting it between each pair of nodes. Choose best.

Neighbourhood



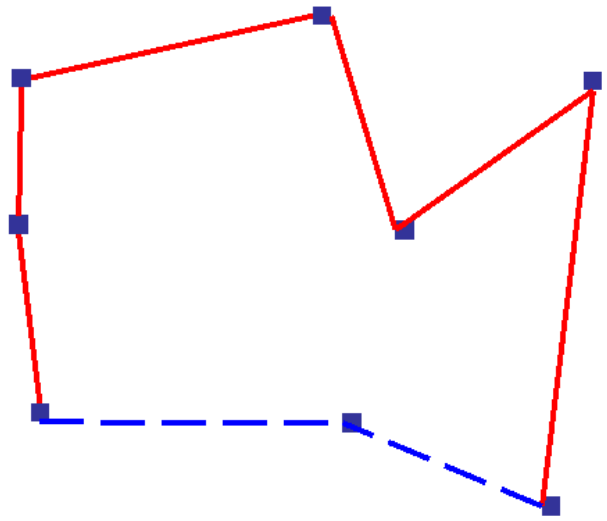
Remove a node. Test inserting it between each pair of nodes. Choose best.

Neighbourhood



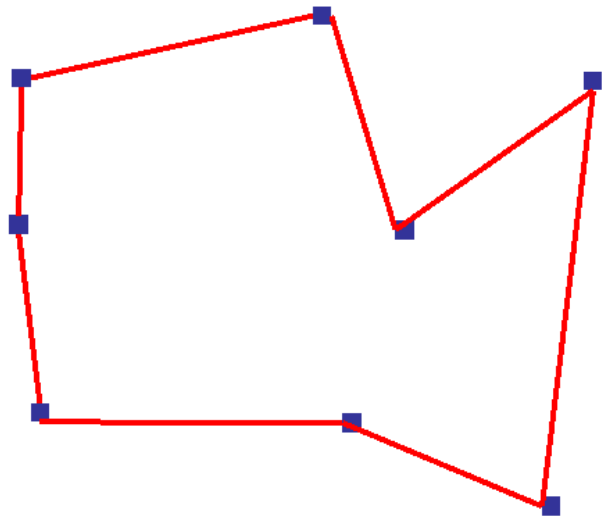
Remove a node. Test inserting it between each pair of nodes. Choose best.

Neighbourhood



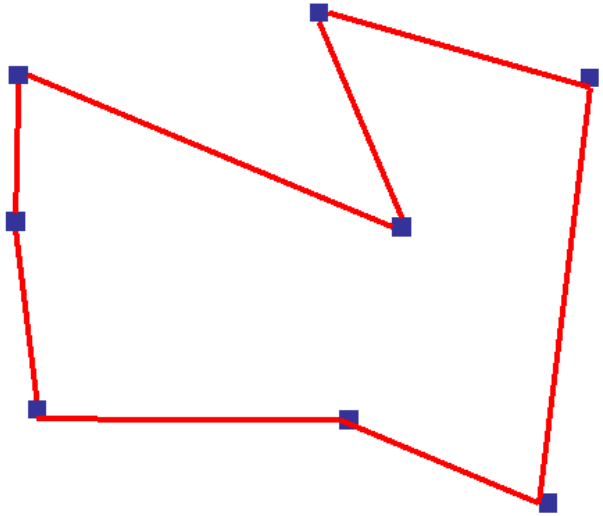
Remove a node. Test inserting it between each pair of nodes. Choose best.

Neighbourhood



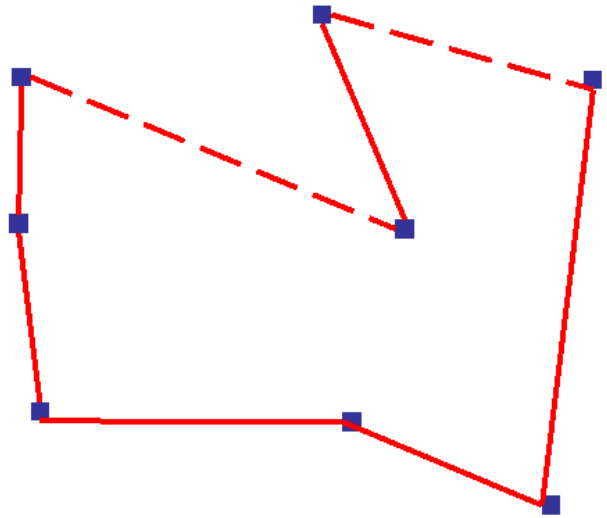
Remove a node. Test inserting it between each pair of nodes. Choose best.

Neighbourhoods



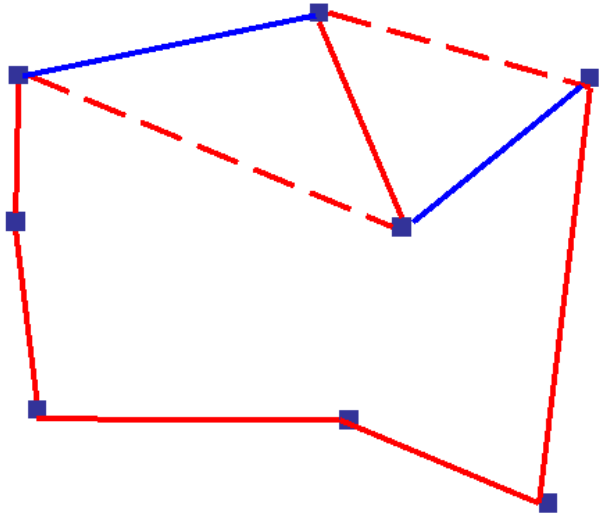
Two-opt neighbourhood

Neighbourhoods



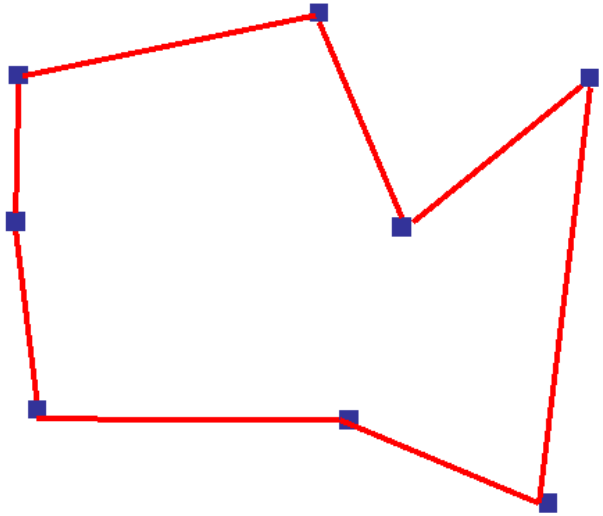
Two-opt neighbourhood - remove 2 arcs

Neighbourhoods



Two-opt neighbourhood - remove 2 arcs, and replace with two others

Neighbourhoods

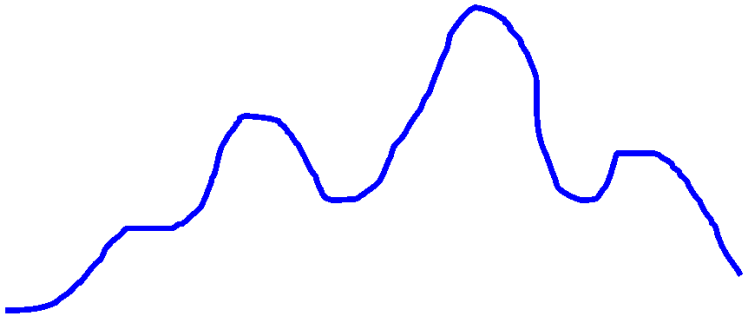


Two-opt neighbourhood - remove 2 arcs, and replace with two others

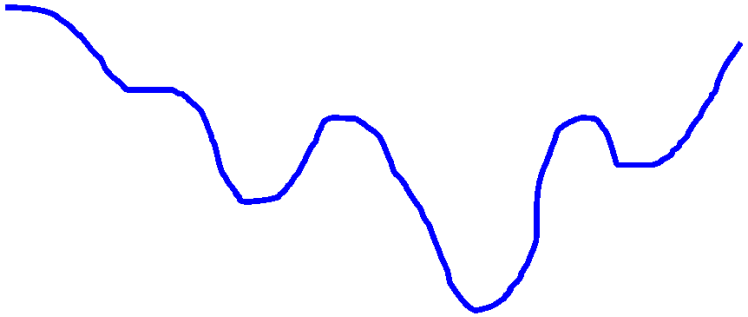
Neighbourhoods

3-opt (n^3 search space), 4-opt etc can also be defined.

Accept - Hill-Climbing



Accept - Hill-Climbing



Hill-climbing search

- It is simply a loop that continually moves in the direction of increasing value-that is, uphill.
- It terminates when it reaches a "peak" where no neighbor has a higher value.
- The algorithm does not maintain a search tree.
- Hill-climbing does not look ahead beyond the immediate neighbors of the current state.
- This resembles trying to find the top of Mount Everest in a thick fog while suffering from amnesia.

Hill-climbing (or gradient ascent/descent)

"Like climbing Everest in thick fog with amnesia"

```
function HILL-CLIMBING( problem) returns a state that is a local maximum
    inputs: problem, a problem
    local variables: current, a node
                     neighbor, a node

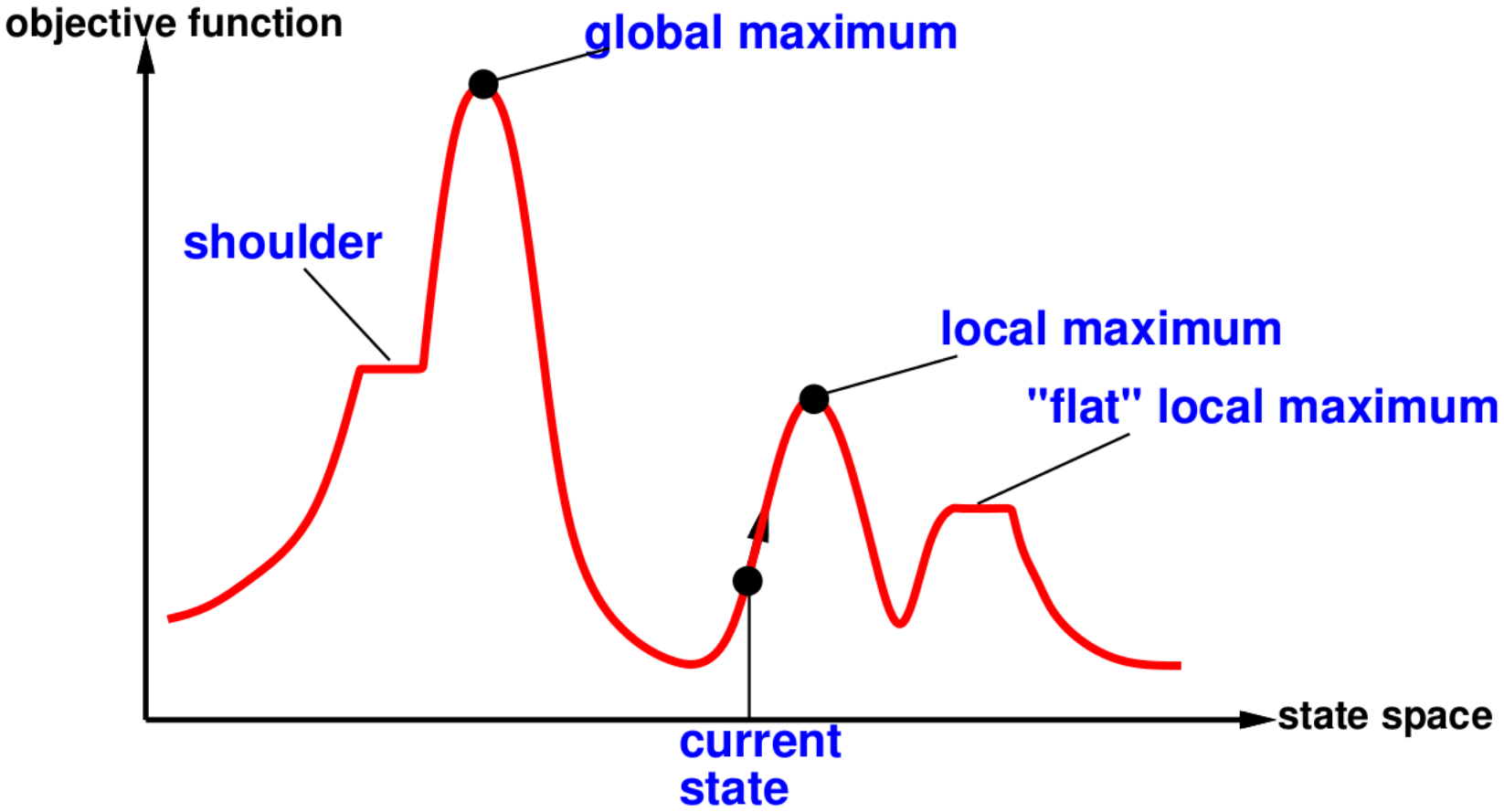
    current ← MAKE-NODE(INITIAL-STATE[problem])
    loop do
        neighbor ← a highest-valued successor of current
        if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
        current ← neighbor
    end
```

At each step, move to a neighbor of higher value
in hopes of getting to a solution having the highest possible value

Can easily modify this for problems where we want to minimize rather than maximize

Hill-climbing, continued

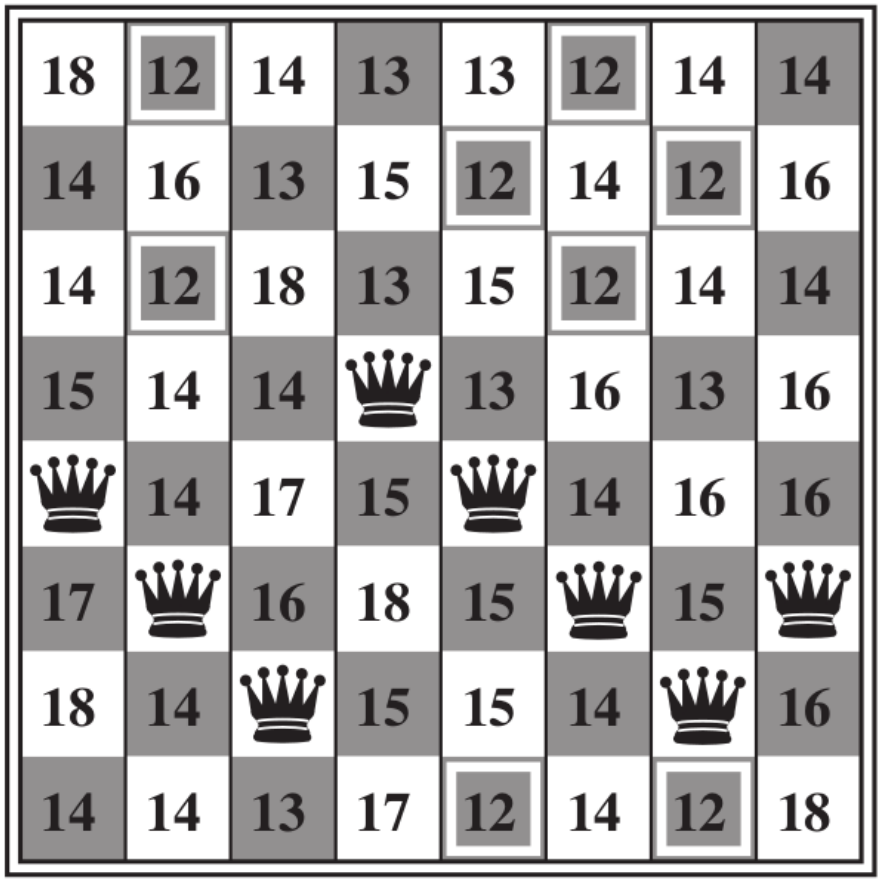
Useful to consider *state space landscape*



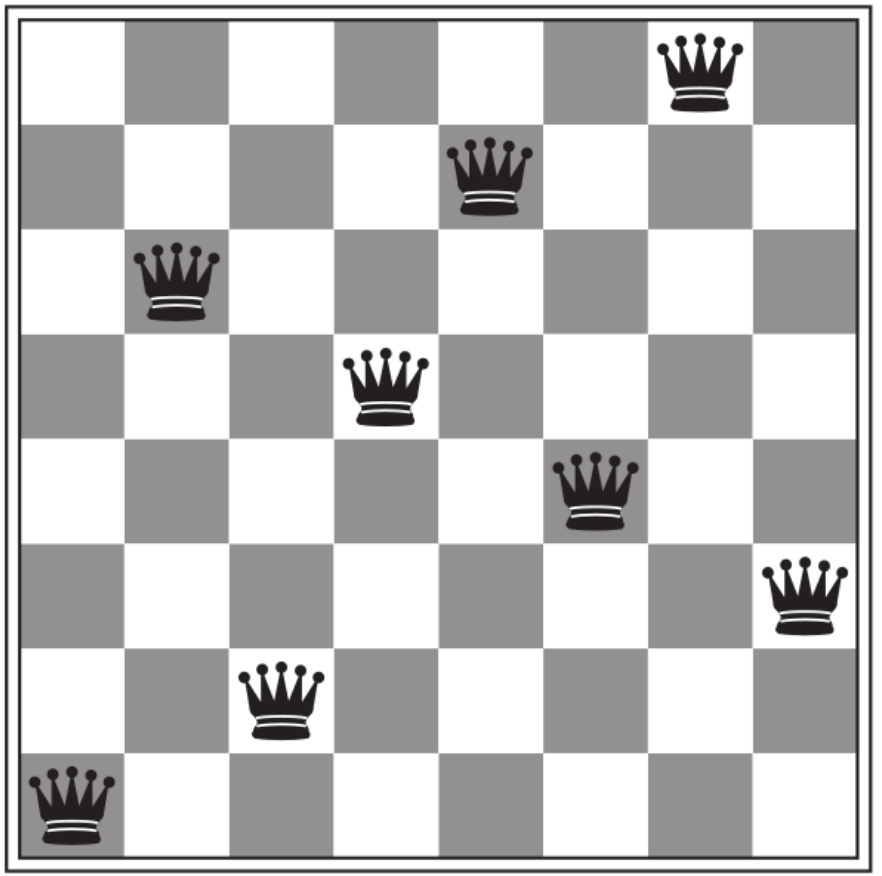
Random-restart hill climbing: repeat with randomly chosen starting points

Russell & Norvig say it's trivially complete; they're almost right

If finitely many local maxima, then $\lim_{\text{restarts}\to\infty} P(\text{complete}) = 1$



(a)

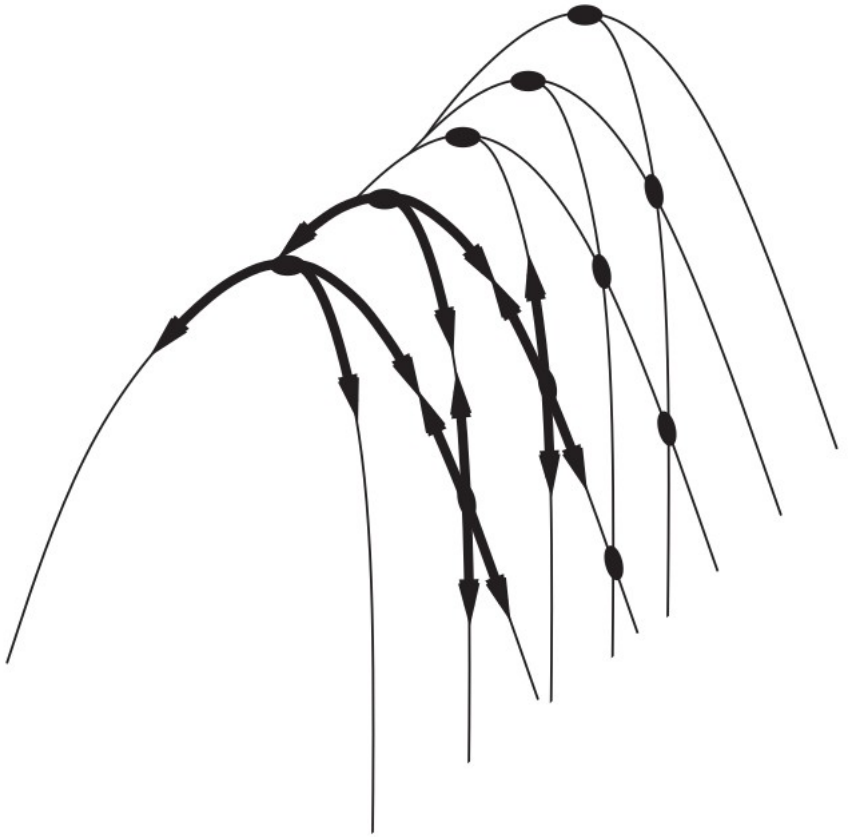


(b)

Figure 4.3 (a) An 8-queens state with heuristic cost estimate $h = 17$, showing the value of h for each possible successor obtained by moving a queen within its column. The best moves are marked. (b) A local minimum in the 8-queens state space; the state has $h = 1$ but every successor has a higher cost.

Ridges: a ridge is shown in Figure 4.4. Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.

Plateaux: a plateau is a flat area of the state-space landscape. It can be a flat local maximum, from which no uphill exit exists, or a **shoulder**, from which progress is possible. (See Figure 4.1.) A hill-climbing search might get lost on the plateau.



In each case, the algorithm reaches a point at which no progress is being made. Starting from a randomly generated 8-queens state, steepest-ascent hill climbing gets stuck 86% of the time, solving only 14% of problem instances. It works quickly, taking just 4 steps on average when it succeeds and 3 when it gets stuck—not bad for a state space with $8^8 \approx 17$ million states.

The algorithm in Figure 4.2 halts if it reaches a plateau where the best successor has the same value as the current state. Might it not be a good idea to keep going—to allow a **sideways move** in the hope that the plateau is really a shoulder, as shown in Figure 4.1? The answer is usually yes, but we must take care. If we always allow sideways moves when there are no uphill moves, an infinite loop will occur whenever the algorithm reaches a flat local maximum that is not a shoulder. One common solution is to put a limit on the number of consecutive sideways moves allowed. For example, we could allow up to, say, 100 consecutive sideways moves in the 8-queens problem. This raises the percentage of problem instances solved by hill climbing from 14% to 94%. Success comes at a cost: the algorithm averages roughly 21 steps for each successful instance and 64 for each failure.

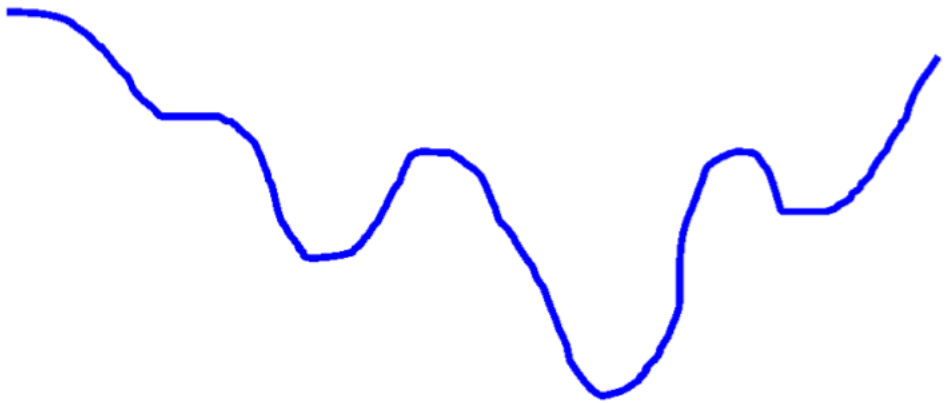
Accept - Hill-Climbing

Greedy Local Search

Systematically search the Neighbourhood. Accept the {first or best} cost-decreasing move found.

Stochastic Local Search

Choose randomly from amongst the cost-decreasing moves found.



- Random-restart hill climbing adopts the well-known adage, “If at first you don’t succeed, try, try again.”
- It conducts a series of hill-climbing searches from randomly generated initial states, 1 until a goal is found.

Meta-Heuristics

To escape a local minimum *sometimes* you want to accept a cost-increasing move

Meta-heuristics are methods (heuristics) for combining other heuristics in a hopefully effective way.

Look at a few: Beam Search, Simulated Annealing, Tabu Search and Genetic Programming

Simulated annealing

Idea: escape local maxima by allowing some "bad" moves
but gradually decrease their size and frequency

function SIMULATED-ANNEALING(*problem, schedule*) **returns** a solution state
  inputs: *problem*, a problem
    schedule, a mapping from time to "temperature"
  local variables: *current*, a node
    next, a node
    T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[*problem*])
  for $i \leftarrow 1$ **to** ∞ **do**
    $T \leftarrow$ *schedule*$[i]$
    if $T = 0$ **then return** *current*
    next ← a randomly selected successor of *current*
    $\Delta E \leftarrow$ VALUE[*next*] – VALUE[*current*]
    if $\Delta E > 0$ **then** *current* ← *next*
    else with probability $e^{\Delta E/T}$, set *current* ← *next*

Sometimes you want to accept a cost-increasing move. Simulated Annealing does this with a probabilistic accept:

$$P(\text{accept}) = e^{-\Delta/T}$$

T (temperature) is a system parameter. As the system "cools" $(T \to 0)$, the probability of accepting a bad move decreases.

Simulated Annealing

Annealing is the process of heating and cooling a metal to change its internal structure for modifying its physical properties. When the metal cools, its new structure is seized, and the metal retains its newly obtained properties. In simulated annealing process, the temperature is kept variable.

We initially set the temperature high and then allow it to 'cool' slowly as the algorithm proceeds. When the temperature is high, the algorithm is allowed to accept worse solutions with high frequency.

- Widely used in VLSI layout, airline scheduling, etc

Local beam search

```
function BEAM-SEARCH( problem, k) returns a solution state
    start with k randomly generated states
    loop
        generate all successors of all k states
        if any of them is a solution then return it
        else select the k best successors
```

Not the same as k parallel searches
Searches that find good states will recruit other searches to join them

Problem: often all k states end up on same local hill

Stochastic beam search:
choose k successors randomly, biased towards good ones

Close analogy to natural selection

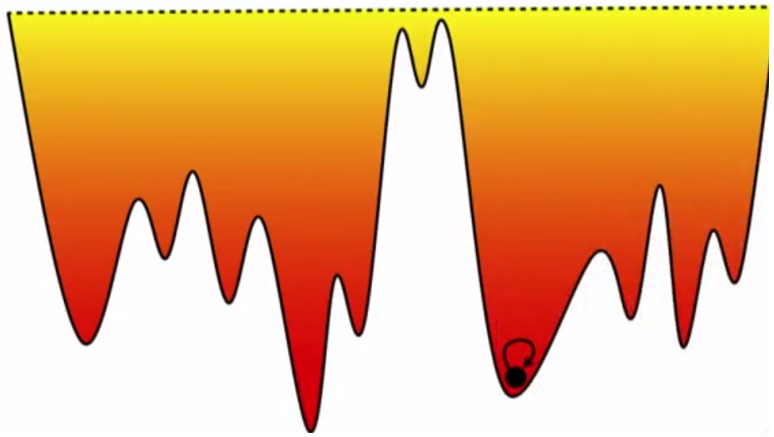
Tabu Search

Basic idea:

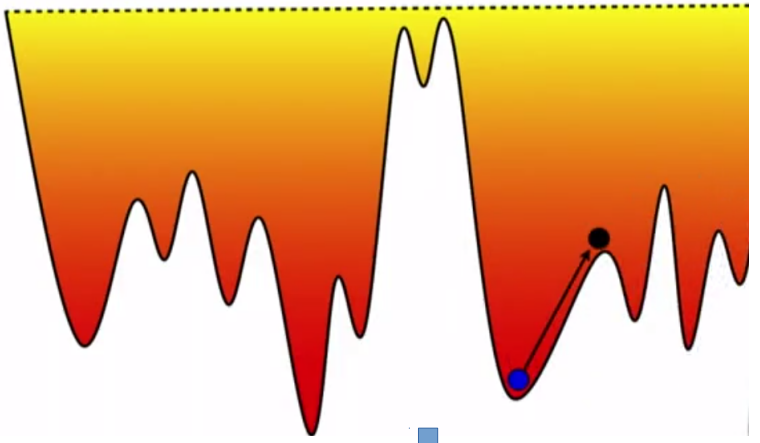
- ♢ Find local minimum
- ♢ Choose and make a cost-increasing move
- ♢ Place the *reverse* move on a *tabu list* for some number of iterations (*tabu tenure* parameter)
- ♢ Repeat

Lots of tweaks, wrinkles and enhancements.

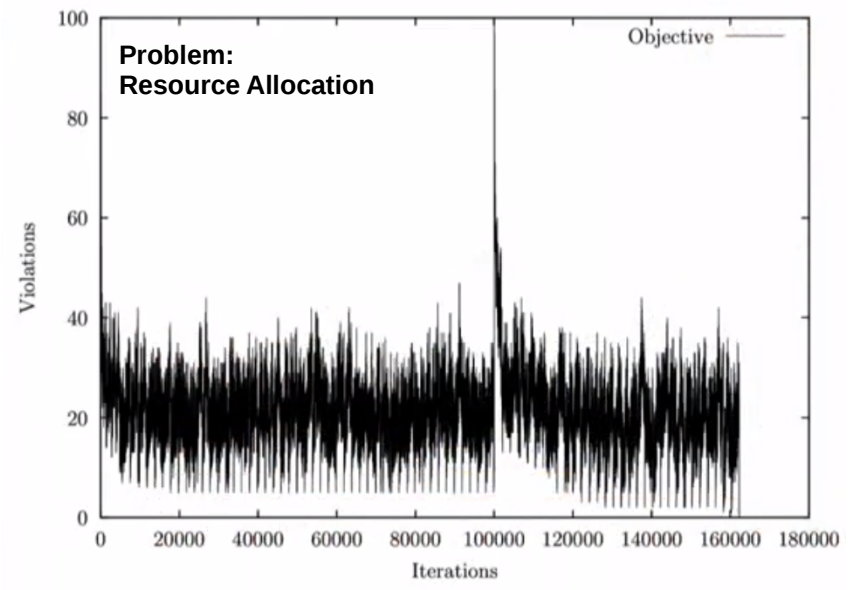
Tabu search is at least as successful as Simulated Annealing.

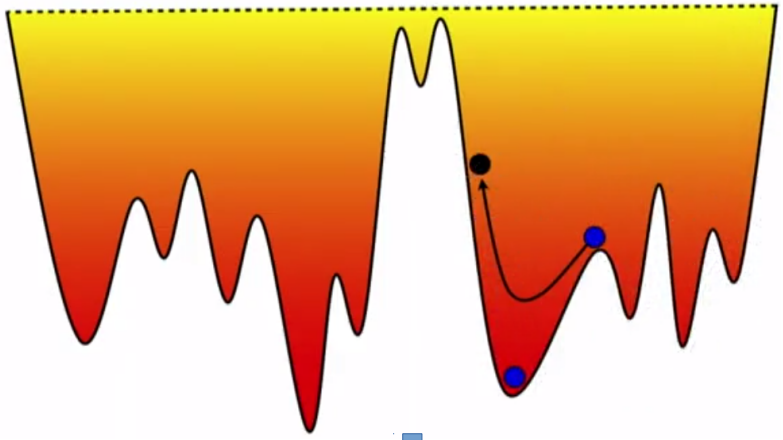




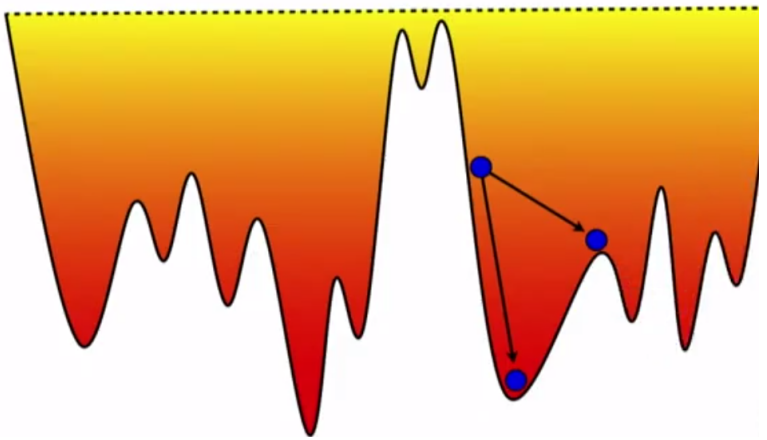




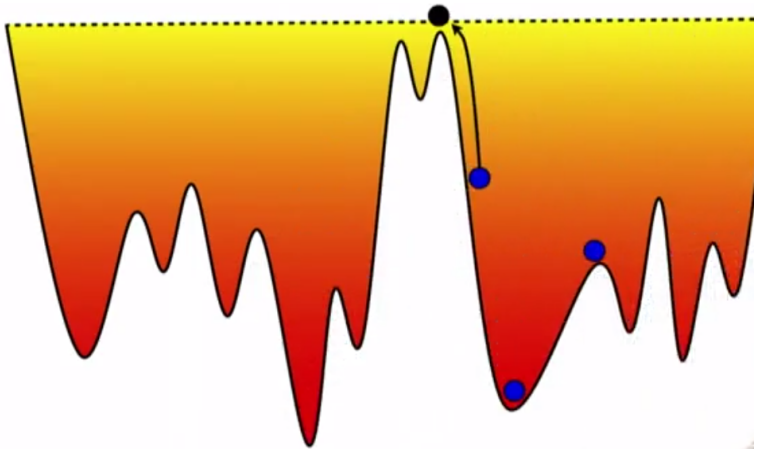

Problem:
Resource Allocation
Objective
Violations
Iterations
0
20
40
60
80
100
0
20000
40000
60000
80000
100000
120000
140000
160000
180000











Other meta-heuristics

There are a host of other meta-heuristics - many based on "biological analogues"

- ♢ Ant Colony Optimisation
- ♢ Bee analogues
- ♢ Memetic programming (== GA + Local Search)

And others

- ♢ Large Neighbourhood Search
- ♢ Variable Neighbourhood Search
- ♢ Problem Space Search
- ♢ Guided Local Search